St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2022-2023
Class: BE-ITA/B, Semester: VII
Subject: Data Science Lab

Experiment – 4

1. **Aim:** To implement a Cognitive Computing Application

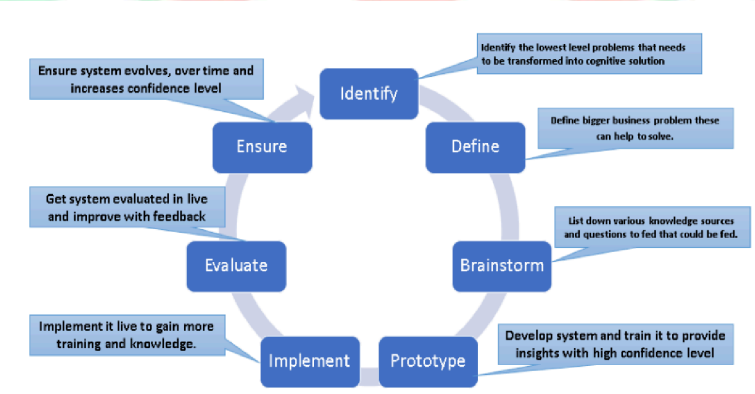
2. **Objectives:** Students should be able to design a solution for a problem using Cognitive Computing.

3. **Prerequisite:** Python basics

4. **Requirements:** PC, Python 3.9, Windows 10/ MacOS/ Linux, IDLE IDE

5. **Pre-Experiment Exercise:**
   Theory:
   - The Cognitive is the mental action to learning and acquiring through thought, experience and the senses.
   - Cognitive computing is computerized model that simulates human thought process in complex situations where the answer may be ambiguous and uncertain.
   - Cognitive computing systems can recognize, understand, analyze, memorize and take out best possible result as or near about the human brain.
   - The basic idea behind this type of computing is that to develop the computer system(include hardware and software) who interacts with human like humans.
   - To accomplish this, cognitive computing makes use of AI and underlying technologies.
   - If you look at cognitive computing as an analog to the human brain, you need to analyze in context all types of data, from structured data in databases to unstructured data in text, images, voice, sensors, and video.

Design Principles of Cognitive Computing:



Phases in NLP:

Phonological Analysis:
- It is applied if input is speech.

Morphological Analysis
- Deals with understanding distinct words according to their morphemes.
- Eg: Unhappiness: broken down into three morphemes (prefix, stem, suffix).
- Stem is considered as free morpheme and prefix and suffix are considered are

bound morphemes.

Lexical Analysis:

- Lexicon of a language means the collection of words and phrases in the language.
- Lexical analysis is dividing the whole chunk of text into paragraphs, sentences and words.
- Lexicon normalization is often needed in Lexical analysis.
- The most common lexicon normalization are:
    - Stemming: it is a rudimentary rule based process of stripping the suffixes. From word.
    - Lemmatization: organized procedure of obtaining the root form of the word by using dictionary and morphological analysis.

Syntactic Analysis:

- Deals with analyzing the words of a sentence so as to uncover the grammatical structure of the sentence.
- Eg: “Colorless green idea”
- Checked for dependency grammar and parts of speech tags .

Semantic Analysis:

- Determines possible meaning of the sentence by focusing on the interactions among word level meanings in the sentence.

Discourse Integration:

- Focuses on the properties of the text as a whole that convey meaning by making connections between component sentences.

Pragmatic Analysis:

- Explains how extra meaning is read into texts without actually being encoded in them.
- It helps user to discover intended effect by applying set of rules that characterize cooperative dialogues.

6. Laboratory Exercise

A. Procedure

i. Use google colab for programming.
ii. Import nltk package.
iii. Demonstrate all phases of NLP on a given text.
iv. Add relevant comments in your programs and execute the code. Test it for various cases.

7. Post-Experiments Exercise:

A. Extended Theory:

a. Explain design Principles of Cognitive Computing.

B. Post Lab Program:

a. Select a application of your choice in domain like health care, banking, finance and implement

C. Conclusion:

1. Write what was performed in the program (s) .
2. What is the significance of program and what Objective is achieved?

D. References:

[1]Judith S. Hurwitz, Marcia Kaufman, Adrian Bowles, “Cognitive Computing and Big Data Analytics”, Wiley India, 2015.

Laboratory Exercise
Use google colab for programming.
Import nltk package.
Demonstrate all phases of NLP on a given text.
Add relevant comments in your programs and execute the code. Test it for various cases.

1) TOKENIZATION

Tokenizing text

```
import nltk
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('tagsets')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('tagsets_json')
nltk.download('stopwords')
```

```
[2] from nltk.tokenize import sent_tokenize, word_tokenize
    nltk.download('punkt')
    example_text = "I want to be a certified artificial intelligence professional"
    print('sentence-->', sent_tokenize(example_text))
    print('word-->' ,word_tokenize(example_text))
    for i in word_tokenize(example_text):
      print(i)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
sentence--> ['I want to be a certified artificial intelligence professional']
word--> ['I', 'want', 'to', 'be', 'a', 'certified', 'artificial', 'intelligence', 'professional']
I
want
to
be
a
certified
artificial
intelligence
professional
```

2) Normalization and Lemmatization

Normalization - Stemming and Lemmatization

```
[ ] #### Stemming
```

```
[3] from nltk.stem import PorterStemmer
    #from nltk.tokenize import word_tokenize
    ps = PorterStemmer()
    #example_words = ["python","pythoner","pythoning","pythoned","pythonic"]
    example_words = "Indices"
    print(ps.stem(example_words))
    #for w in example_words:
      #print(ps.stem(w))
```

```
indic
```

```
#### Lemmatization
```

```
#nltk.download('wordnet')
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
print("rocks when lemmatized :",   lemmatizer.lemmatize("rocks"))
print("corpora when lemmatized :", lemmatizer.lemmatize("corpora"))

#ps = PorterStemmer()
#print("rocks when Stemmed :", ps.stem("rocks"))
#print("corpora when Stemmed :", ps.stem("corpora"))

# a denotes adjective in "pos"
print("better :", lemmatizer.lemmatize("better", pos ="a"))
```

```
rocks when lemmatized : rock
corpora when lemmatized : corpus
better : good
```

Parts of Speech Tagging

```
example_text = "The training is going great and the day is very fine.The code is working and all are happy about it"
# this has to run first time
token = nltk.word_tokenize(example_text)
nltk.pos_tag(token)  # error
  # this has to run first time

# We can get more details about any POS tag using help funciton of NLTK as follows.
nltk.help.upenn_tagset("PRP$")
nltk.help.upenn_tagset("JJ$")
nltk.help.upenn_tagset("VBG")
```

```
PRP$: pronoun, possessive
    her his mine my our ours their thy your
JJ: adjective or numeral, ordinal
    third ill-mannered pre-war regrettable oiled calamitous first separable
    ectoplasmic battery-powered participatory fourth still-to-be-named
    multilingual multi-disciplinary ...
VBG: verb, present participle or gerund
    telegraphing stirring focusing angering judging stalling lactating
    hankerin' alleging veering capping approaching traveling besieging
    encrypting interrupting erasing wincing ...
```

bigrams/trigrams/ngrams

```
word_data = 'I want to be a certified artificial intelligence professional'
nltk_tokens = nltk.word_tokenize(word_data)
#print(list(nltk.bigrams(nltk_tokens)))
#nltk_tokens = nltk.word_tokenize(example_text)
print('Bigram-->',list(nltk.bigrams(nltk_tokens)))
print('-----------------------------------------------------')
print('Trigram-->',list(nltk.trigrams(nltk_tokens)))
print('-----------------------------------------------------')
print('5-gram-->',list(nltk.ngrams(nltk_tokens,5)))
```

```
Bigram--> [('I', 'want'), ('want', 'to'), ('to', 'be'), ('be', 'a'), ('a', 'certified'), ('certified', 'artificial'), ('artificial', 'intelligence'), ('intelligence', 'professional')]
-----------------------------------------------------
Trigram--> [('I', 'want', 'to'), ('want', 'to', 'be'), ('to', 'be', 'a'), ('be', 'a', 'certified'), ('a', 'certified', 'artificial'), ('certified', 'artificial', 'intelligence'), ('artificial', 'intelligenc
-----------------------------------------------------
5-gram--> [('I', 'want', 'to', 'be', 'a'), ('want', 'to', 'be', 'a', 'certified'), ('to', 'be', 'a', 'certified', 'artificial'), ('be', 'a', 'certified', 'artificial', 'intelligence'), ('a', 'certified', 'a
```

Printing all combinations of n-grams

```
import nltk
from nltk.util import ngrams
def word_grams(words, min=1, max=5):
    s = []
    for n in range(min, max):
        for ngram in ngrams(words, n):
            s.append(' '.join(str(i) for i in ngram))
    return s
print(word_grams(nltk_tokens))
```

```
['I', 'want', 'to', 'be', 'a', 'certified', 'artificial', 'intelligence', 'professional', 'I want'
```

Removing stop words

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

#example_text = "This is an example showing off stop word filtration."
example_text = 'Manoj want to be a certified artificial intelligence professional'
stop_words = set(stopwords.words("english"))
print("List of the Stop words=",stop_words)
print("------------------------------------------------------------")

words = word_tokenize(example_text)

filtered_sentence = []

for w in words:
  if w not in stop_words:
    filtered_sentence.append(w)

print("Words after stopword removal--",filtered_sentence)
```

```
List of the Stop words= {'re', 'each', 'whom', 'its', 'again', 'a', 'been', 'didn', 'i', 'who', 'up', 'ma', 'yours', 'having',
------------------------------------------------------------
Words after stopword removal-- ['Manoj', 'want', 'certified', 'artificial', 'intelligence', 'professional']
```

Tokenization and Stemming

```
new_text = "It is very important to be pythonic while you are pythoning with python.Python name is derived from the pythons"
words=word_tokenize(new_text)
for w in words:
  print(ps.stem(w))
```

```
it
is
veri
import
to
be
python
while
you
are
python
with
python.python
name
is
deriv
from
the
python
```

Parts of Speech Tagging

```
[ ] example_text = "The training is going great and the day is very fine.The code is working and all are happy about it"
    #nltk.download('averaged_perceptron_tagger') # this has to run first time
    token = nltk.word_tokenize(example_text)
    nltk.pos_tag(token)  # error
    #nltk.download('tagsets')  # this has to run first time

    # We can get more details about any POS tag using help funciton of NLTK as follows.
    nltk.help.upenn_tagset("PRP$")
    nltk.help.upenn_tagset("JJ$")
    nltk.help.upenn_tagset("VBG")
```

```
PRP$: pronoun, possessive
    her his mine my our ours their thy your
JJ: adjective or numeral, ordinal
    third ill-mannered pre-war regrettable oiled calamitous first separable
    ectoplasmic battery-powered participatory fourth still-to-be-named
    multilingual multi-disciplinary ...
VBG: verb, present participle or gerund
    telegraphing stirring focusing angering judging stalling lactating
    hankerin' alleging veering capping approaching traveling besieging
    encrypting interrupting erasing wincing ...
```

Named Entity Recognition using Spacy

```
import spacy
#from spacy import displacy
from collections import Counter
import en_core_web_sm # en_core_web_md and en_core_web_lg
import pprint
# Run in console  python -m spacy download en_core_web_sm / en_core_web_md / en_core_web_lg
nlp = en_core_web_sm.load()
doc = nlp(u'European authorities fined Google a record $5.1 billion on Wednesday for abusing its power in the mobile phone market and ordered the company to alter its practices')
print([(X.text, X.label_) for X in doc.ents])
print([(X, X.ent_iob_, X.ent_type_) for X in doc])

sentences = [x for x in doc.ents]
print(sentences)
#displacy.serve(nlp(str(sentences)), style='ent')

#--You can view visualization at: http://localhost:5000/---#
#--displacy.render(nlp(str(sentences)), style='ent') will give HTML Code --#
```

```
[('European', 'NORP'), ('Google', 'ORG'), ('$5.1 billion', 'MONEY'), ('Wednesday', 'DATE')]
[(European, 'B', 'NORP'), (authorities, 'O', ''), (fined, 'O', ''), (Google, 'B', 'ORG'), (a, 'O', ''), (record, 'O', ''), ($, 'B', 'MONEY'), (5.1, 'I', 'MONEY'), (billion, 'I', 'MONEY')
[European, Google, $5.1 billion, Wednesday]
```

```
!pip install gensim
```

```
import gensim
from gensim import corpora
from pprint import pprint

# How to create a dictionary from a list of sentences?
documents = ["Saudi Arabia has warned that in the event of no action been taken against Iran",
             "If the world does not take a strong and firm action to find alternatives of crude oil",
             "oil prices will jump to unimaginably high numbers."]

documents_2 = ["Automobile fuel prices in India have been rising."
               " With India being the world's third largest oil importer,"
               " a record gain in crude oil prices could also aggravate "
               "India's fiscal situation and make it tougher"
               "for the government to combat a slowdown in economic growth."]

# Tokenize(split) the sentences into words
texts = [[text for text in doc.split()] for doc in documents]

# Create dictionary
dictionary = corpora.Dictionary(texts)

# Get information about the dictionary
print(dictionary)
```

```
Dictionary<35 unique tokens: ['Arabia', 'Iran', 'Saudi', 'action', 'against']...>
```

TF-IDF using gensim

```
from gensim import models
import numpy as np

documents = ["This is the first line",
             "This is the second sentence",
             "This third document"]

# Create the Dictionary and Corpus
mydict = corpora.Dictionary([simple_preprocess(line) for line in documents])
corpus = [mydict.doc2bow(simple_preprocess(line)) for line in documents]

# Show the Word Weights in Corpus
for doc in corpus:
    print([[mydict[id], freq] for id, freq in doc])

# Create the TF-IDF model
tfidf = models.TfidfModel(corpus, smartirs='ntc')

# Show the TF-IDF weights
for doc in tfidf[corpus]:
    print([[mydict[id], np.around(freq, decimals=2)] for id, freq in doc])
```

```
[['first', 1], ['is', 1], ['line', 1], ['the', 1], ['this', 1]]
[['is', 1], ['the', 1], ['this', 1], ['second', 1], ['sentence', 1]]
[['this', 1], ['document', 1], ['third', 1]]
[['first', 0.63], ['is', 0.31], ['line', 0.63], ['the', 0.31], ['this', 0.13]]
[['is', 0.31], ['the', 0.31], ['this', 0.13], ['second', 0.63], ['sentence', 0.63]]
[['this', 0.15], ['document', 0.7], ['third', 0.7]]
```

```
import gensim.downloader as api
#api.info('glove-wiki-gigaword-50')
dir(api)
import gensim
dir(gensim)
```

```
['__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 '__path__',
 '__spec__',
 '__version__',
 '_matutils',
 'corpora',
 'downloader',
 'interfaces',
 'logger',
 'logging',
 'matutils',
 'models',
 'parsing',
 'similarities',
 'topic_coherence',
 'utils']
```

```
[ ] # create Topic Models with LDA
```

```
[ ] dataset = api.load("text8")
    dataset = [wd for wd in dataset]
```

```
[==================================================] 100.0% 31.6/31.6MB downloaded
```

```
from gensim.utils import simple_preprocess
from smart_open import smart_open
import os

# Create gensim dictionary form a single text file
dictionary = corpora.Dictionary(simple_preprocess(line, deacc=True) for line in open('text8', encoding='utf-8'))
dictionary
```

```
<gensim.corpora.dictionary.Dictionary at 0x7c525f0623d0>
```

```
[ ] # doc to bag of words
```

```
[ ] # List with 2 sentences
    my_docs = ["Who let the dogs out?","Who? Who? Who? Who?"]

    # Tokenize the docs
    tokenized_list = [simple_preprocess(doc) for doc in my_docs]

    # Create the Corpus
    mydict = corpora.Dictionary()
    mycorpus = [mydict.doc2bow(doc, allow_update=True) for doc in tokenized_list]
    pprint(mycorpus)
```

```
[[(0, 1), (1, 1), (2, 1), (3, 1), (4, 1)], [(4, 4)]]
```

Post Experiment Exercise

Select a application of your choice in domain like health care, banking, finance and implement Spam Text prediction using SVM

```
import pandas as pd
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords
from nltk import pos_tag, word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn import svm
from sklearn.metrics import confusion_matrix

data = pd.read_csv("spam.csv", encoding = "latin-1")
data = data[['v1', 'v2']]
data = data.rename(columns = {'v1': 'label', 'v2': 'text'})

lemmatizer = WordNetLemmatizer()
stopwords = set(stopwords.words('english'))

def review_messages(msg):
    # converting messages to lowercase
    msg = msg.lower()
    return msg

def alternative_review_messages(msg):
    # convert to lowercase
    msg = msg.lower()

    # tokenize words
    words = word_tokenize(msg)
```

```
    # get POS tags
    nltk_pos = [tag[1] for tag in pos_tag(words)]

    # convert nltk POS tags to WordNet POS tags
    wnpos = ['a' if tag.startswith('J') else
             'v' if tag.startswith('V') else
             'n' if tag.startswith('N') else
             'r' if tag.startswith('R') else 'n'
             for tag in nltk_pos]

    # lemmatize
    words = [lemmatizer.lemmatize(word, pos=wnpos[i]) for i, word in enumerate(words)]

    # remove stopwords
    words = [word for word in words if word not in stopwords]

    return "".join(words)

# Processing text messages
#data['text'] = data['text'].apply(alternative_review_messages)
data['text'] = data['text'].apply(review_messages)


# train test split
X_train, X_test, y_train, y_test = train_test_split(data['text'], data['label'],
test_size = 0.1, random_state = 1)

# training vectorizer
vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(X_train)

# training the classifier
svm = svm.SVC(C=1000)
svm.fit(X_train, y_train)

# testing against testing set
X_test = vectorizer.transform(X_test)
y_pred = svm.predict(X_test)
cm=confusion_matrix(y_test, y_pred)
print(cm)
tn, fp, fn, tp = cm[0][0], cm[0][1], cm[1][0], cm[1][1]
print("Accuracy: ", ((tp+tn)/(tp+tn+fp+fn))*100)
print("Precision: ", (tp/(tp+fp))*100)
print("Recall: ", (tp/(tp+fn))*100)
print("F1 Score: ", (2*tp/(2*tp+fp+fn))*100)

# test against new messages
def pred(msg):
    msg = vectorizer.transform([msg])
    prediction = svm.predict(msg)
    return prediction[0]
```

```
[[490   0]
 [  5  63]]
Accuracy:  99.10394265232975
Precision:  100.0
Recall:  92.64705882352942
F1 Score:  96.18320610687023
```

Confusion Matrix output on SVM training

Testing for unseen msg (First 10 are ham, Next 10 are spam)

```
messages = [
    "Hey, are we still meeting at 6 tonight?",
    "Don't forget to bring the documents for tomorrow.",
    "Happy birthday! Wish you all the best 🎉",
    "I'll call you back after my class.",
    "Can you send me the assignment notes?",
    "Let's grab lunch at the new cafe.",
    "I reached home safely, don't worry.",
    "Good morning, hope you have a great day!",
    "Please confirm if the parcel arrived.",
    "Thanks for helping me yesterday.",
    "Congratulations! You have won a free iPhone. Click here to claim.",
    "WINNER!! You have been selected for a $1000 Walmart gift card.",
    "Get cheap loans at 0% interest. Call now!",
    "Exclusive deal! Buy 1 get 1 free on all products. Limited time!",
    "You have been pre-approved for a credit card. Apply today.",
    "URGENT! Your account has been suspended. Verify at this link.",
    "Claim your free vacation trip to Bali now. Reply YES to book.",
    "Lowest insurance rates guaranteed. Don't miss out.",
    "You are selected for a cash prize. Call 1800-123-456 immediately.",
    "Click this link to unlock premium content for FREE!"
]

hamCt=0
spamCt=0
for indx,msg in enumerate(messages):
  if pred(msg)=="ham" and indx<10:
    hamCt=hamCt+1
  elif pred(msg)=="spam" and indx>=10:
    spamCt=spamCt+1
  print(msg)
  print(f"category={pred(msg)}",end="")
  print("\n")

print(f"Ham accuracy:{(hamCt/10)*100}")
print(f"Spam accuracy: {(spamCt/10)*100}")
```

OUTPUT:

```
1. Hey, are we still meeting at 6 tonight?
category=ham

2. Don’t forget to bring the documents for tomorrow.
category=ham

3. Happy birthday! Wish you all the best 🎉
category=ham

4. I’ll call you back after my class.
category=ham

5. Can you send me the assignment notes?
category=ham

6. Let’s grab lunch at the new cafe.
category=ham

7. I reached home safely, don’t worry.
category=ham

8. Good morning, hope you have a great day!
category=ham

9. Please confirm if the parcel arrived.
category=ham

10. Thanks for helping me yesterday.
category=ham
```

All Ham Messages detected correctly

```
11. Congratulations! You have won a free iPhone. Click here to claim.
category=spam

12. WINNER!! You have been selected for a $1000 Walmart gift card.
category=ham

13. Get cheap loans at 0% interest. Call now!
category=ham

14. Exclusive deal! Buy 1 get 1 free on all products. Limited time!
category=ham

15. You have been pre-approved for a credit card. Apply today.
category=ham

16. URGENT! Your account has been suspended. Verify at this link.
category=ham

17. Claim your free vacation trip to Bali now. Reply YES to book.
category=spam

18. Lowest insurance rates guaranteed. Don’t miss out.
category=ham

19. You are selected for a cash prize. Call 1800-123-456 immediately.
category=spam

20. Click this link to unlock premium content for FREE!
category=spam
```

Only few spam messages predicted correctly

```
Ham accuracy:100.0
Spam accuracy: 40.0
```

OVERALL ACCURACY 70 percent